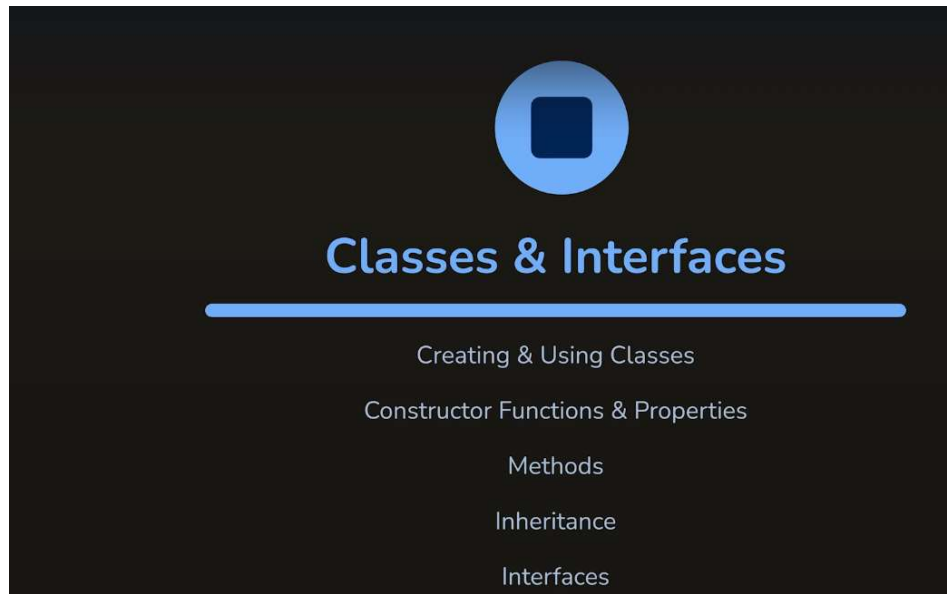
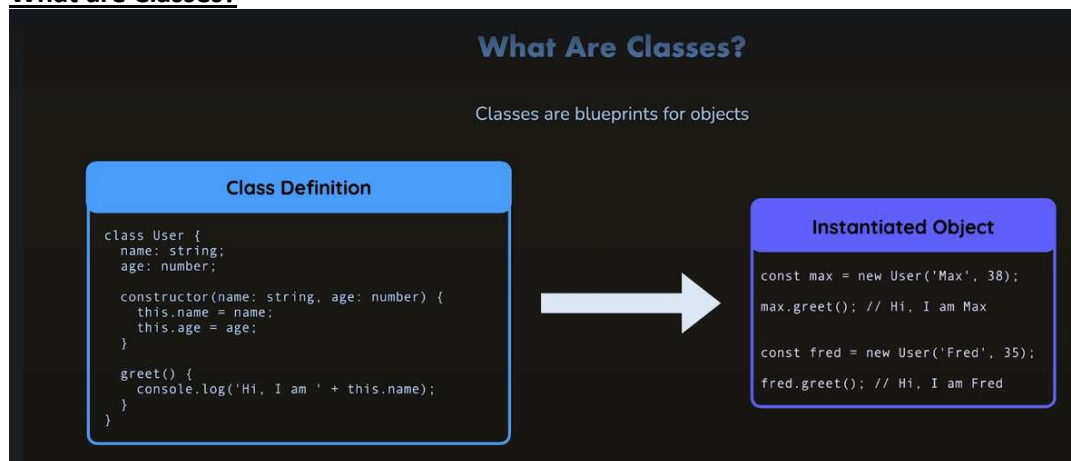


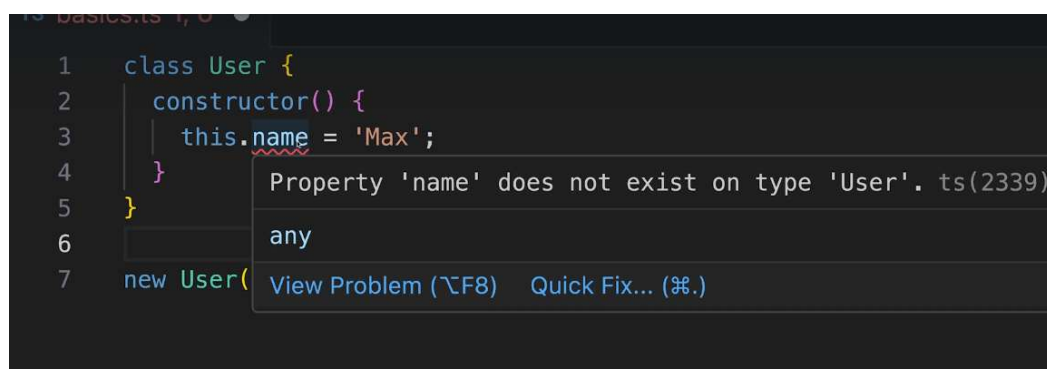


What are Classes?



Creating a First Class

This is ok in vanilla js but it produces error in typescript. Because in ts, properties are added in a different way.



A Useful TypeScript Shortcut & Compiling to JavaScript

```

class User {
  name: string;
  constructor User(n: string, a: number): User
  constructor(n: string, a: number) {
    // this.name = 'Max';
    this.name = n;
    this.age = a;
  }
}

new User()

```

<v Maximilian>If you were using vanilla JavaScript,</v>

```

class User {
  constructor(n: string, a: number) {
    // this.name = 'Max';
    this.name = n;
    this.age = a;
  }
}

new User()

```

the code would instead look like this.

TypeScript offers a convenient shortcut.

but alternatively, you could also create it

with a little shortcut where you don't create

```

class User {
  name: string;
  age: number;
  constructor(n: string, a: number) {
    // this.name = 'Max';
    this.name = n;
    this.age = a;
  }
}

```

those properties up here,

but where you instead add a certain keyword

in front of your parameters here in the constructor.

And that keyword, which you wanna add,

is the public keyword in front of a

By writing 'public' we tell typescript to immediately create a property in that class that has the same name as this parameter behind the scene also assign value that it receives on this parameter.

```
class User {  
  constructor(public n: string, public a: number) {  
    // this.name = 'Max';  
    this.name = n;  
    this.age = a;  
  }  
}
```

new User()

to immediately create a property in that class

```
class User {  
  constructor(public n: string, public a: number) {  
  }  
}  
  
new User()
```

Making Sense of "public" and "private"

```

1
2 class User {
3     public hobbies: string[] = [];
4
5     constructor(public name: string, public age: number) {}
6 }
7
8 const max = new User('Max', 36);
9 const fred = new User('Fred', 29);
10
11 max.age = 37;
12
13 console.log(max, fred);

```

So I can assign a new value to age like this,

If you were to mark age as private with the private keyword,

```

11
12 class User {
13     public hobbies: string[] = [];
14
15     constructor(public name: string, private age: number) {}
16 }
17
18 const max = new User('Max', 36);
19 const fred = new User('Fred', 29);
20
21 max.age = 37;
22
23 console.log(max, fred);

```

which is another keyword offered by TypeScript,

```

11
12 class User {
13     public hobbies: string[] = [];
14
15     constructor(public name: string, private age: number) {}
16 }
17
18 const max = new User('Max', 36);
19 const fred = new User('Fred', 29);
20
21 max.age = 37;
22
23 console.log(fred);

```

then as you can see, you would get an error down here

```

11
12 class User {
13     public hobbies: string[] = [];
14
15     constructor(public name: string, private age: number) {}
16 }
17
18 const max = new User('Max', 36);
19 const fred = new User('Fred', 29);
20
21 console.log(max.age);
22
23 console.log(max);

```

but printing it like this is also not allowed.

Instead, private properties can only be accessed

from inside the class, for example, from inside a method

```

class User {
    public hobbies: string[] = [];

    constructor(public name: string, private age: number) {}

    greet() {
        console.log('My age: ' + this.age);
    }
}

const max = new User('Max', 36);
const fred = new User('Fred', 29);

```

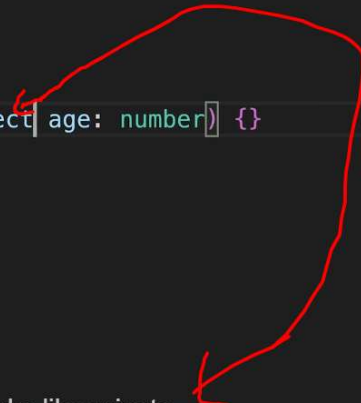
the age property with help of the this keyword.

Besides public and private

```

1  // }
2
3  class User {
4      public hobbies: string[] = [];
5      ⚡
6      constructor(public name: string, protected age: number) {}
7
8      greet() {
9          console.log('My age: ' + this.age);
10     }
11 }
12
13 const max = new User('Max', 36);
14 const fred = new User('Fred', 36);
15
16 // there also is protected, which works like private,

```



```

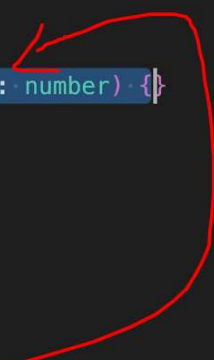
10 // }
11
12 class User {
13     public hobbies: string[] = [];
14     ⚡
15     constructor(public name: string, protected age: number) {}
16
17     greet() {
18         console.log('My age: ' + this.age);
19     }
20 }
21
22 const max = new User('Max', 36);
23 const fred = new User('Fred', 36);
24
25 // may be accessed in classes that inherit from this class,

```

```

11
12 class User {
13     public hobbies: string[] = [];
14     ⚡
15     constructor(public name: string, private age: number) {}
16
17     greet() {
18         console.log('My age: ' + this.age);
19     }
20 }
21
22 const max = new User('Max', 36);
23 const fred = new User('Fred', 36);
24
25 // The shortcut is nice to have,

```



ne but the main idea is that you can control

how these properties may be accessed outside of the class,

public and private are TypeScript exclusive features.

These keywords do not exist in vanilla JavaScript.

Private properties are counterparts of the regular class properties which are public, including [class fields](#), class methods, etc. Private properties get created by using a hash # prefix and cannot be legally referenced outside of the class. The privacy encapsulation of these class properties is enforced by JavaScript itself. The only way to access a private property is via [dot notation](#), and you can only do so within the class that defines the private property.

Private properties were not native to the language before this syntax existed. In prototypal inheritance, its behavior may be emulated with [weakrefs](#), objects or [closures](#), but they can't compare to the # syntax in terms of ergonomics.

Syntax

```
JS
class ClassWithPrivate {
  #privateField;
  #privateFieldWithInitializer = 42;

  #privateMethod() {
    // ...
  }

  static #privateStaticField;
  static #privateStaticFieldWithInitializer = 42;

  static #privateStaticMethod() {
    // ...
  }
}
```

Indeed, vanilla JavaScript does have the concept of private properties as well.

of private properties as well.

By default, all properties are public,

but it does have the concept of private properties as well.

to the # syntax in terms of ergonomics.

Syntax

```
JS
class ClassWithPrivate {
  #privateField;
  #privateFieldWithInitializer = 42;

  #privateMethod() {
    // ...
  }

  static #privateStaticField;
  static #privateStaticFieldWithInitializer = 42;

  static #privateStaticMethod() {
    // ...
  }
}
```

There are some additional syntax restrictions:

- All private identifiers declared within a class must be unique. The namespace is shared between static and instance properties. The only exception is when the two declarations define a getter-setter pair.
- The private identifier cannot be `#constructor`.

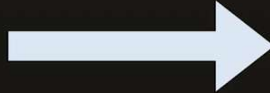
Describe With help of this hashtag syntax,

Introducing Interfaces

Object type definitions & contracts that can be implemented by classes

Interface Definition

```
interface Authenticatable {  
  email: string;  
  password: string;  
  
  login(): void;  
  logout(): void;  
}
```



Usage as Object Type

```
let authUser: Authenticatable;  
  
authUser = {  
  email: 'test@example.com',  
  password: 'abc1',  
  login() { ... },  
  logout() { ... }  
};
```

Implementation as Contract

```
class User implements Authenticatable {  
  constructor(  
    public email: string,  
    public password: string  
  ) {}  
  
  login() { ... }  
  
  logout() { ... }  
}
```

Ensuring Base Types with Interfaces

```
20   logout() {  
21     // ...  
22   }  
23 }  
24  
25 // ...  
26  
27 function authenticate(user: Authenticatable) {}  
28  
29 // type Authenticatable = {  
30 //   role: string;  
31 // }
```

ng;
You are guaranteed to get a user object

that has a login method

string;
because that is clearly defined by this interface.